

Verifiable Frontends

Dapp frontends served from on-chain storage — and verified in the user's own client.

Fredrik & the Access Layer cluster · August 2026

github.com/ethstorage/web3-url-verifier · [web3-verifier-extension](#)

Agenda

- 01 Our approach — and what already runs
- 02 WEBCAT, side by side
- 03 Where we could take part

The problem

- Trust stops at the contracts — the frontend is just a hosted file
- Compromise it and funds drain, **however sound the contracts are**
- The user has no way to tell

The idea

- Put the frontend on-chain, addressed by `web3://` — [ERC-4804](#)
- Same keys as the contracts → **trust root already in place**
- Light client checks the bytes against chain state
- The gateway stays in the path, but drops out of the trust model

It works today

- **End to end** — our CLI fetches, Colibri verifies, result reported
- Two storage models, one verification path
Ethereum-native · vitalikblog.eth / [EthStorage](#) + L1 · eth-store.eth
- Chrome extension built — **every response body is reachable**, via CDP
- Verification not yet wired into it. That's next.

On WEBCAT

- [1TS grant](#) announced 5 August — good news for the problem
docs.webcat.tech · github.com/freedomofpress/webcat
- Their client **already does light-client verification**
- Same shape as ours — the difference is **how much has to be built underneath**

On WEBCAT — side by side

	WEBCAT	OURS
The registry behind the name	A purpose-built permissioned chain, updated by oracles that fetch <code>enrollment.json</code> over DNS/HTTPS	ENS — already a registry on Ethereum, nothing to build
Who may publish	Signers + threshold in <code>enrollment.json</code>	The content contract's own access control — its owner / multisig
How the client checks it	Light client on that chain	Light client on Ethereum (Colibri)
Where the content lives	The site's own server	On-chain — Ethereum-native or EthStorage
What it protects	Integrity	Integrity, plus availability

On WEBCAT — a specialization

- A DNS name forces the registry. Off-chain content forces the manifest.
- ENS points at the contract — **both layers collapse**
- Nothing to stand up, nothing to keep honest
- And availability: one origin server, vs. anyone can serve it

What we'd like to discuss

If we want to take an active part in this — what would you suggest?

01 The ERC

Don't hard-code a single registry. — Where should we plug in?

02 Colibri licensing

We can take it to [corpus-core](#) ourselves. — Is anyone on your side already in touch?

03 Anything else you'd suggest

Work underway we should know about, people we should talk to — or somewhere you'd push us instead.